

Incident Report — Telegram Code Delivery Failure

TScopier Production — User ruthdaberechi2100@gmail.com

DATE

September 1, 2026

STATUS

RESOLVED

SEVERITY

HIGH

DURATION

~10 hours (06:29 – 17:58 UTC)

COMPONENT

Worker — AuthService / SessionManager

ROOT CAUSE

Stuck in-memory guard flag

1. Executive Summary

A user was completely blocked from receiving Telegram verification codes for over 10 hours due to a stuck in-memory state in the listener process. A single `send_code` request hung mid-execution, leaving a guard flag permanently set. Every subsequent request — 50 total — was instantly rejected. The issue was resolved by restarting the listener, and a code fix with TTL + timeout safeguards has been implemented and deployed.

2. Affected User

FIELD	VALUE
Email	ruthdaberechi2100@gmail.com
User ID	b545f6af-fe3e-46fc-8c70-8923b51e0de3
Account Created	August 31, 2026 20:57 UTC
Last Sign-in	September 1, 2026 15:32 UTC
Phone (redacted)	+729****894

3. Timeline

August 31, 2026 — Initial Auth (Working)

TIME (UTC)	EVENT
21:02:10	First <code>send_code</code> with invalid phone format → <code>PHONE_NUMBER_INVALID</code> . Request cleaned up normally.
21:02:40	Second <code>send_code</code> with correct phone → Success . Telegram delivered code via app.
21:03:33	<code>verify_code</code> → Success . User signed in. Active session created.

September 1, 2026 — Failure Window

TIME (UTC)	EVENT
06:29:53	<code>send_code</code> request entered auth flow. <code>send_code_start</code> logged. No follow-up events ever recorded. Request hung. <code>authInFlight</code> guard permanently set.
06:30 – 15:33	44 consecutive <code>send_code</code> attempts, all rejected with <code>duplicate_in_flight</code> .
06:59 – 12:17	6 QR login attempts, all failed or timed out.
16:53+	Listener in reconnect loop: disconnects and renews lease every ~20 seconds.
17:58	Listener restarted. In-memory state cleared. User can request codes again.

4. Root Cause

The auth flow uses an in-memory `Set<string>` called `authInFlight` to prevent concurrent `send_code` requests. When a request starts, the user ID is added. When it finishes (success or error), the `finally` block removes it.

The Problem

At 06:29:53, a `send_code` request added the user ID to `authInFlight`, then hung inside `sessionManager.prepareForAuth()`. The `finally` block never ran. The guard flag was stuck forever — with no TTL, no expiry, and no cleanup path.

Why It Hung

`prepareForAuth()` stops the live listener and waits for the session lease to be released (polling up to 45 seconds). The listener was in a reconnect loop — disconnecting and re-acquiring the lease every ~20 seconds. This prevented the lease from ever being truly released, causing the wait to deadlock.

Why It Was Catastrophic

- **No TTL on `authInFlight`** — a stuck entry blocked the user permanently.
- **No timeout on `sendCode`** — a hung `prepareForAuth` kept the coroutine alive forever.
- **No self-healing cleanup** — the periodic timer only cleaned up the `pending` Map, not `authInFlight`.

5. The Fix

1 Operational Fix — Listener Restart

Restarted the "TScopier - Listener" service on Railway (deployment `0f776eca`, 17:58 UTC). Cleared the in-memory state. User could request codes again immediately.

2 Code Fix — TTL on `authInFlight` (2 minutes)

Added `authInFlightTimestamps` map and `isAuthInFlight()` helper. Entries older than 2 minutes are evicted as stale. Applied to all guard checks in `sendCode`, `resendCode`, `startQrLogin`, `getQrStatus`, and the auth guard callback.

3 Code Fix — 90-Second Operation Timeout

Wrapped the `prepareForAuth` → `client.connect` → API call chain in `Promise.race` with a 90-second timeout. If the inner promise hangs, the timeout fires, the `finally` block runs, and `authInFlight` is cleaned up automatically.

4 Code Fix — Periodic Stale Cleanup

Added stale `authInFlight` eviction to the existing 60-second cleanup timer as a safety net. Entries older than the TTL are removed even if no new request triggers the guard.

6. Files Changed

FILE	CHANGE
<code>worker/src/authService.ts</code>	TTL constants, timestamp map, <code>isAuthInFlight()</code> / <code>markAuthInFlight()</code> / <code>clearAuthInFlight()</code> helpers, <code>Promise.race</code> timeout wrappers in <code>sendCode</code> and <code>startQrLogin</code> , stale eviction in cleanup timer, updated all guard call sites

7. Verification

- ✓ TypeScript build passes (`npm --prefix worker run build`)
- ✓ All existing `authService` tests pass (redaction: 8/8, resend: 10/10)
- ✓ Listener restarted at 17:58 UTC — user confirmed able to request codes
- ✓ Code review verdict: PASS_WITH_NOTES (no critical or high findings)
- ✓ Deployed to staging and dev (origin + upstream)

8. Deployment Status

TARGET	BRANCH	COMMIT	STATUS
Origin staging	staging	<code>69940fd0</code>	PUSHED
Origin dev	dev	<code>2d4c5cb0</code>	PUSHED

Upstream staging	staging	69940fd0	PUSHED
Upstream dev	dev	9bb70efb	PUSHED

9. Follow-ups

- Deploy code fix to production after staging validation.
- Consider adding `AbortController` support to fully cancel the losing `Promise.race` branch (currently the background work continues until cleanup timer fires — bounded and self-healing, but not instant).
- Investigate the listener reconnect loop for this user — it was the underlying trigger for the lease deadlock.